

Analiza repozitorija

monter-balkonskih-ograj

Monter Ograj Pro - Profesionalna tablicna aplikacija za monterje
balkonskih ograj

GitHub: <https://github.com/markec12345678/monter-balkonskih-ograj>

Datum analize: 5. junij 2026

Avtor analize: Z.ai

Kazalo vsebine

1. Povzetek projekta	4
2. Tehnicni sklad in arhitektura	4
2.1. Gradbene tehnologije	4
2.2. Arhitektura MVVM	4
3. Struktura repozitorija	5
4. Podatkovni model in Room baza	6
4.1. Entiteta Project	6
5. Ključne funkcionalnosti	7
5.1. Upravljanje projektov	7
5.2. Skiciranje in vizualizacija ograj	7
5.3. Integracija z Gemini AI	7
5.4. Simuliran LiDAR skener	8
5.5. Kalkulator cen in ROKSAL katalog	8
6. Ocena kakovosti kode	8
6.1. Prednosti	8
6.2. Slabosti in izboljšave	9
7. Varnostna analiza	9
8. Predlogi za izboljšave	10

8.1. Kriticne izboljšave	10
8.2. Pomembne izboljšave	10
8.3. Zelene izboljšave	11

9. Zakljucenek	11
-----------------------	-----------

1. Povzetek projekta

Repozitorij **monter-balkonskih-ograj** vsebuje izvorno kodo Android aplikacije z imenom **Monter Ograj Pro**. Gre za profesionalno tablicno aplikacijo, namenjeno monterjem balkonskih ograj, ki delujejo na terenu po Sloveniji. Aplikacija omogoča terenske meritve, skiciranje ograj na fotografijah balkonov, vizualni predogled različnih stilov ograj v živo ter avtomatsko generiranje strokovnih poročil s pomočjo umetne inteligence (Google Gemini AI). Projekt je bil ustvarjen s pomočjo orodja Google AI Studio in uporablja arhitekturo MVVM z lokalno Room podatkovno bazo, Jetpack Compose za uporabniški vmesnik ter Retrofit za komunikacijo z Gemini API-jem.

Aplikacija je jasno usmerjena v slovenski trg - ves uporabniški vmesnik je v slovenscini, podprte so slovenske gradbene norme (višina ograj, RAL barve, davčne stopnje DDV), vgrajen pa je tudi katalog izdelkov slovenskega proizvajalca ROKSAL iz Kranja. Projekt cilja na tablicne računalnike (detekcija zaslonke širine nad 680dp za master-detail razporeditev), hkrati pa podpira tudi mobilne telefone z animiranimi prehodi med zasloni.

2. Tehnicni sklad in arhitektura

2.1. Gradbene tehnologije

Komponenta	Tehnologija	Različica
Jezik	Kotlin	2.2.10
UI Okvir	Jetpack Compose + Material 3	BOM 2024.09.00
Podatkovna baza	Room (SQLite)	2.7.0
Mrežna knjižnica	Retrofit + OkHttp + Moshi	2.12.0 / 4.10.0
AI Integracija	Google Gemini 3.5 Flash API	v1beta
Kamera	CameraX (camera2)	1.5.0
Slikovni nalagalek	Coil Compose	2.7.0
Gradnja	Gradle (Kotlin DSL) + KSP	AGP 9.1.1
Min SDK	Android 7.0 (API 24)	-
Target SDK	Android 16 (API 36)	-

Tabela 1: Tehnološki sklad projekta

2.2. Arhitektura MVVM

Aplikacija sledi vzorcu MVVM (Model-View-ViewModel), ki je priporočljivi arhitekturni vzorec za moderne Android aplikacije. Struktura je razdeljena na tri plasti: podatkovno (data), logično (ViewModel) in predstavno (UI/Compose). Podatkovna plast vključuje Room entiteto **Project**, DAO vmesnik **ProjectDao**, konverterje tipov **Converters** (Moshi za JSON serializacijo seznama SketchLine), singleton podatkovno bazo **AppDatabase** ter repozitorij **ProjectRepository**, ki abstrahira dostop do DAO-ja. ViewModel plast upravlja s stanjem aplikacije preko StateFlow objektov, omogoča pa dodajanje, posodabljanje, brisanje projektov ter shranjevanje slik in zagon analize Gemini AI.

3. Struktura repozitorija

Repozitorij sledi standardni strukturi Android projekta z Gradle Kotlin DSL konfiguracijo. Izvorna koda je organizirana v paketu com.example z jasno ločnico med podatkovno plastjo (data), poslovno logiko (ui) in uporabniskimi komponentami (ui/screens, ui/components, ui/theme). Naslednja tabela prikazuje ključne datoteke in njihove namene.

Datoteka	Namen
MainActivity.kt	Vstopna točka aplikacije, inicializacija Room DB, ViewModel-ja in Compose vsebine
data/Models.kt	Podatkovni modeli: FloatPoint, SketchLine, Project (Room entiteta)
data/ProjectDao.kt	Room DAO vmesnik s CRUD operacijami in Flow poizvedbami
data/AppDatabase.kt	Room podatkovna baza (singleton), različica 1, destruktivna migracija
data/TypeConverters.kt	Moshi konverter za serializacijo List<SketchLine> v JSON
data/ProjectRepository.kt	Repozitorij - abstrakcija nad DAO, izpostavlja Flow seznamov
ui/ProjectViewModel.kt	ViewModel z StateFlow stanji, CRUD logika, shranjevanje slik, Gemini klici
ui/GeminiClient.kt	Retrofit klient za Gemini 3.5 Flash API zBase64 kodiranjem slik
ui/screens/DashboardScreen.kt	Glavni zaslon s seznamom projektov, master-detail razporeditvijo, obrazci in kalkulatorjem
ui/components/RailingVisualizer.kt	Kanvas za skiciranje ograj na slikah z DrawingCanvas, prikaz ograd v živo, kalibracija, primerjava Prej/Potem
ui/components/LidarScannerDialog.kt	Simuliran LiDAR/AR skener z animiranimi učinki, postavljanjem sidrnih točk in uvozom skenov
ui/components/LidarCalibrationDialog.kt	Kalibracijski dialog z simuliranim napredkom v treh korakih

ui/components/PhotoCaptureComponent.kt	Komponenta za zajem fotografije balkona preko kamere
ui/components/ImageAnnotationCanvas.kt	Kanvas za anotacijo slik z locnikom in povečevalnim steklom
ui/components/RoksalCatalog.kt	Brskalnik po ROKSAL katalogu izdelkov s tehničnimi listi in navodili

Tabela 2: Ključne datoteke repozitorija

4. Podatkovni model in Room baza

Jedro podatkovnega modela je Room entiteta **Project**, ki shrani vse podatke o posameznem projektu montaže ograje. Entiteta vsebuje osnovne podatke o stranki (ime, naslov, telefon), tehnicne specifikacije ograje (stil, dolzina, visina, globina, nacin montaže, barva RAL), vizualne podatke (pot do originalne slike, seznam skicirnih crt SketchLine) ter rezultate AI analize (geminiEstimate). Vsak projekt ima tudi oznako isFinished za sledenje statusa.

4.1. Entiteta Project

Polje	Tip	Privzeto	Opis
id	Long	auto	Avtomatsko generiran primarni ključ
customerName	String	-	Ime in priimek stranke
address	String	-	Naslov lokacije montaže
railingStyle	String	MODERN_ALU	Stil ograje (5 tipov)
lengthCm / heightCm / widthCm	Int	350/110/0	Dimenzije v centimetrih
mountType	String	V tla	Nacin sidranja (v tla ali bocno)
colorHex / colorName	String	#2D2D2D	Hex barva in RAL oznaka
sketchLines	List<SketchLine>	emptyList()	Seznam skicirnih crt (JSON v bazi)
geminiEstimate	String?	null	Rezultat Gemini AI analize
isFinished	Boolean	false	Status zaključenosti projekta

Tabela 3: Polja entitete Project

Podatkovna baza uporablja Moshi za pretvorbo seznama SketchLine v JSON niz, kar omogoča shranjevanje kompleksnih podatkovnih struktur v SQLite stolpec. Razred SketchLine vsebuje seznam točk (FloatPoint), heksadecimalno barvo, debelino crtice, tip crtice (DRAWING za meritve ali RAILING za vizualizacijo ograje), stil ograje in opcionalno oznako. Ta pristop je priljubljen v manjših projektih, vendar prinaša tudi slabosti: poizvedbe po posameznih skicnih crtah niso mogoče brez nalozitve celotnega seznama, prav tako ni mogoče učinkovito iskati projektov po stilu ograje znotraj skicnih crt.

5. Ključne funkcionalnosti

5.1. Upravljanje projektov

Aplikacija omogoča ustvarjanje novih projektov z vnosom podatkov o stranki (ime, naslov, telefonska številka). Projekti so prikazani v seznamu s hitrim iskanjem po imenu stranke ali naslovu. Vsak projekt lahko označimo kot zaključen ali ga znova odpremo, predvsem pa ga lahko izbrisemo. Na tablicnih računalnikih je uporabljen master-detail razporeditveni vzorec (35:65 razmerje), kjer je seznam projektov na levi strani, podrobnosti pa na desni. Na mobilnih napravah so sezname in podrobnosti prikazani sekvenčno z animiranimi prehodi (slide + fade).

5.2. Skiciranje in vizualizacija ograj

Srediska funkcionalnost aplikacije je **RailingVisualizer**, ki omogoča nalaganje fotografije balkona in nanjo skiciranje ograj v realnem času. Komponenta podpira tri načine risanja: VIEW (pregled), MEASURE (meritve z oznakami dolzin) in RAILING (vizualizacija izbranega stila ograje na sliki). Uporabnik lahko izbira med petimi stili ograj ROKSAL: H-Line (vodoravne letve), Steklena (glass), V-Line (pokončne letve), Panelna (CNC laserski izrez) in Klasik (tradicijska kovana). Na voljo je sedem RAL barv (antracit, črna, bela, srebrna eloksirana, rjava, imitacija hrast, imitacija oreh) z vizualnim izbirnikom.

Napredne funkcije vključujejo mrežno prikazovanje za natančnost, pripenjanje kotov na 45-stopinjske sektorje (angle snapping), umerjanje merila (kalibracija), primerjavo Prej/Potem z drsnim preходом, razdeljeni pogled (split-view) ter razveljavljanje zadnjega koraka. Slike se nalagajo preko fotoaparata ali galerije, pri čemer aplikacija uporablja CameraX za neposreden zajem in FileProvider za varno deljenje URI-jev.

5.3. Integracija z Gemini AI

Aplikacija vključuje globoko integracijo z Google Gemini 3.5 Flash modelom preko REST API-ja. Ko monter vnese vse podatke o projektu in naloži fotografijo balkona, lahko sproži AI analizo, ki generira stiridelno strokovno poročilo v slovenscini: (1) Statika in varnost glede na slovenske predpise, (2) Tehnična kalkulacija materiala (število stebričkov, dolžina ročajev, sidrni vijaki), (3)

Navodila za montažo na terenu in (4) Prodajna predstavitev za stranko. Slika balkona se kompresira na največ 800px in kodira v Base64 za prenos k API-ju. Casovne omejitve HTTP povezave so nastavljene na 60 sekund za vsako od treh operacij (connect, read, write), kar zagotavlja dovolj časa za obdelavo slikovnih podatkov z AI modelom.

5.4. Simuliran LiDAR skener

Ena izmed vizualno najizrazitejših komponent je **LidarScannerDialog**, ki simulira LiDAR/AR skeniranje balkona. Komponenta prikazuje predogled kamere v ozadju z animiranimi učinki: matrika točk z globinskim učinkom, animirana radarska črta, ki se premica po zaslonu, ročno postavljanje 3D sidrskih točk ter samodejno zaznavanje robov balkona. Po končanem "skeniranju" se generirajo prednastavljene SketchLine crte za glavni in stranska robova balkona. Pomembno je poudariti, da gre za **simulacijo** - dejansko LiDAR skeniranje trenutno ni implementirano, vendar je v kodi predvidena integracija z ARCore v prihodnosti (komentar "V prihodnosti bo tu ArFragment"). Komponenta vključuje tudi kalibracijski dialog (LidarCalibrationDialog) s tremi simuliranimi koraki in indikatorjem napredka.

5.5. Kalkulator cen in ROKSAL katalog

V zavijku "Podatki in Meritve" je vgrajen interaktivni kalkulator cen, ki upošteva stil ograje (osnovna cena na meter), dimenzije (z 10-odstotnim dodatkom za odrezek in varnostni dobicek), visinski multiplikator (za ograje višje od 100 cm), način montaze (v tla 50 EUR, bocno v fasado 140 EUR), popust v odstotkih ter slovensko DDV stopnjo (9,5-odstotni za stanovanjske objekte ali 22-odstotni splosni). Katalog ROKSAL izdelkov (CatalogBrowser) prikazuje starih modelov ograj s povezavami na tehnične liste in navodila za montažo, vendar so povezave trenutno referenčne in ne vodijo na dejanske dokumente.

6. Ocena kakovosti kode

6.1. Prednosti

Modularna arhitektura: Koda je jasno razdeljena na podatkovno plast, ViewModel in UI komponente. Uporaba repozitorija kot abstrakcijske plasti nad DAO-jem je dobra praksa, ki olajša morebitne spremembe v podatkovnem viru. ViewModel uporablja StateFlow za reaktivno posodabljanje UI-ja, kar je skladno s priporocili Google za moderno Android arhitekturo.

Tematska podpora: Aplikacija podpira temni in svetli način z dinamičnimi barvami Material You (Android 12+). Uporabljene so komponente Material 3 s pravilnimi barvnimi shemami za oba načina.

Odzivna zasnova: Detekcija tabličnega zaslona (680dp) za master-detail razporeditev kaže pozornost do uporabniške izkušnje na različnih velikostih zaslonov. Animirani prehodi na mobilnih napravah dodajajo poliranost.

Varnost API ključev: Uporaba Secrets Gradle Plugin-ja z .env datoteko za shranjevanje Gemini API ključa prepreči pomotni vpis ključa v izvirno kodo. Prav tako je v kodi preverjanje, da ključ ni prazen ali enak nadomestnemu nizu "MY_GEMINI_API_KEY".

6.2. Slabosti in izboljšave

Velike datoteke komponent: DashboardScreen.kt in RailingVisualizer.kt sta izjemno obsežni (več sto vrstic), kar otežuje vzdrževanje in testiranje. Priporočljiva je razdelitev na manjše, fokusirane komponente. Na primer, obrazec za vnos dimenzij bi lahko bil locena komponenta, kalkulator cen locen od prikaza projekta, priemer Prej/Potem locen od glavnega vizualizatorja.

Ni.Dependency Injection: Aplikacija ne uporablja DI okvira (Hilt, Koin). ViewModel se ročno instancira z ViewModelProvider.Factory v MainActivity, Room podatkovna baza pa se ustvarja neposredno v aktivnosti. Za večji projekt bi bilo vnašanje odvisnosti z Hiltom znatno izboljšanje, saj bi olajšalo testiranje in zmanjšalo tesno sklopljenost med komponentami.

Odsotnost navigacijskega okvira: Kljub temu, da je Navigation Compose deklariran v odvisnostih (komentiran), aplikacija ne uporablja formalnega navigacijskega graficon. Namesto tega stanje upravlja z lastnimi StateFlow spremenljivkami in pogojnim risanjem. To vodi v globoko gnezdenje komponent in zapleteno upravljanje stanja, ki ga navigacijski okvir poenostavi.

Simulirana LiDAR funkcionalnost: LiDAR skener je vizualno impresiven, vendar ne opravi dejanskega 3D skeniranja. Generirane "zaznane" crte so harcodirane in ne temeljijo na dejanski geometriji balkona. Uporabnik bi lahko bil zaveden, da skener dejansko zaznava robove. Priporočljiva je bodisi jasna oznaka "simulacija" bodisi dejanska integracija z ARCore Depth API-jem za naprave, ki ga podpirajo.

Destruktivna migracija baze: AppDatabase uporablja fallbackToDestructiveMigration(), kar pomeni, da bo ob spremembi sheme celotna baza izbrisana. Za produkcijsko aplikacijo je to nesprejemljivo, saj uporabniki izgubijo vse podatke o projektih. Pravilni migracijski strategiji sta podajanje eksplicitnih migracijskih funkcij ali uporaba Automigration.

Pomanjkanje testov: V repozitoriju so le nadomestni testi (ExampleUnitTest, ExampleInstrumentedTest, ExampleRobolectricTest), ki ne pokrivajo dejanske poslovne logike. Za produktivno aplikacijo bi bili potrebni testi za ViewModel, Repository, GeminiClient in ključne UI komponente. Roborazzi za slikovne teste je sicer deklariran, a se ne uporablja.

7. Varnostna analiza

Aplikacija ima nekaj pomembnih varnostnih premislekov, ki jih je treba upoštevati pri nadaljnjem razvoju in uvajanju v produkcijsko okolje. Naslednja tabela povzema ključne varnostne vidike in priporočila.

Vidik	Trenutno stanje	Priporocilo
API ključ	Shranjen v .env, bran preko BuildConfig	Za produkcijo: uporaba stretnega backend proxy-ja, ki skriva API ključ pred klientom
Podatki strank	Shranjeni v lokalni Room DB brez šifriranja	Implementacija SQLCipher za šifriranje baze (osebni podatki)
Mrežna komunikacija	HTTPS za Gemini API, brez potrdilnega pripenjanja	Dodaj Certificate Pinning za Gemini API domen
ProGuard/R8	Izključen (isMinifyEnabled = false)	Vklopi minifikacijo in obfuscacijo za release build
Debug keystore	Vključen v repozitorij (base64)	Odstrani debug keystore iz repozitorija, uporabi CI/CD skrivnosti
Slike projektov	Shranjene v internal filesDir (zaščitene pred drugimi aplikacijami)	Dobro, vendar razmisliti o šifriranju občutljivih fotografij

Tabela 4: Varnostna analiza

8. Predlogi za izboljšave

8.1. Kriticne izboljšave

Implementirati Dependency Injection: Uvedba Hilt okvira za upravljanje odvisnosti bi bistveno izboljšala testabilnost, zmanjšala skopljeno in poenostavila zivljenjski cikel ViewModelov. Namesto ročne izdelave ViewModelFactory-ja v MainActivity bi Hilt samodejno vbrizgaval odvisnosti.

Razdeliti velike komponente: DashboardScreen.kt (vec kot 800 vrstic) in RailingVisualizer.kt (vec kot 700 vrstic) bi morali biti razdeljeni na manjše, fokusirane komponente. Na primer: ProjectFormTab, GeminiReportTab, RailingStyleSelector, ColorPicker, PriceCalculator, SplitViewContrast in podobne podkomponente. To izboljša berljivost, vzdržljivost in omogoča neodvisno testiranje.

Uvesti navigacijski okvir: Odstraniti komentar pri Navigation Compose odvisnosti in implementirati formalni NavGraph. To bo poenostavilo upravljanje stanja, omogocilo deep linking in izboljšalo UX s pravilnim obnavljanjem stanja pri vrtenju zaslona ali sistemskem brisanju procesa.

8.2. Pomembne izboljšave

Implementirati pravilno migracijo baze: Zamenjati fallbackToDestructiveMigration() z eksplicitnimi migracijskimi funkcijami. Celo za enostavne spremembe sheme je pomembno

ohraniti uporabniške podatke. Alternativa je uporaba @AutoMigration iz Room knjižnice za avtomatsko generiranje migracij.

Dodati teste: Implementirati enotske teste za ViewModel, Repository in GeminiClient. Dodati UI teste s Compose Testing knjižnico za ključne poti uporabe (ustvarjanje projekta, skiciranje, zagon AI analize). Roborazzi je že konfiguriran in bi lahko takoj uporabljen za regresijsko testiranje vizualnih komponent.

Dejanska ARCore integracija: Namesto simuliranega LiDAR skenerja implementirati pravo integracijo z ARCore Depth API-jem za naprave, ki ga podpirajo (naprave s ToF senzorjem ali LiDAR-jem). Za naprave brez podpore prikazati jasno obvestilo o omejitvi in ne navidezno funkcionalno simulacijo.

8.3. Zelene izboljšave

Podpora za deljenje projektov: Omogočiti izvoz projekta (slika, skice, AI poročilo) v PDF ali deljivo povezavo. Trenutno ni mogoče deliti rezultatov s stranko ali sodelavci brez fizičnega posredovanja tablice. Ustvarjanje PDF poročila z vsemi podatki bi bilo naravna razširitev obstoječe Gemini analize.

Offline sinhronizacija: Dodati podporo za sinhronizacijo projektov z oblacnim strežnikom (Firebase Firestore ali lasten backend). Trenutno so vsi podatki lokalni, kar pomeni, da izguba naprave pomeni izgubo vseh podatkov. Sinhronizacija bi omogočila tudi delo v timu z več monterji.

Večjezična podpora: Lokalizirati aplikacijo z uporabo Androidovih string resources. Trenutno so vsi nizi harcodirani v Kotlin kodi, kar otežuje morebitno širitev na druge trge. Slovenci govorijo več jezikov, toda sosednje trge (Hrvaska, Avstrija, Italija) bi bilo mogoče pokriti z relativno malo napora.

9. Zaključek

Projekt Monter Ograj Pro predstavlja ambiciozen in vizualno izrazit pristop k digitalizaciji terenskega dela monterjev balkonskih ograj. Aplikacija združuje več naprednih tehnologij (Jetpack Compose, Room, CameraX, Gemini AI, simuliran LiDAR) v koherentno celoto, ki jasno naslavlja potrebe slovenskega trga. Uporabniški vmesnik je poliran z animacijami, master-detail razporeditvijo za tablice in tematsko podporo Material You.

Kljub vizualni privlačnosti pa projekt potrebuje strukturnejši pristop k arhitekturi in kakovosti kode. Ključne pomanjkljivosti so: odsotnost Dependency Injection okvira, prevelike komponente, ki bi jih bilo treba razdeliti, odsotnost formalne navigacije, simulirana LiDAR funkcionalnost, ki bi lahko zavedla uporabnike, ter destruktivna migracija podatkovne baze, ki je nesprejemljiva za produkcijsko uporabo. Prav tako je kritično pomanjkanje testov, ki bi zagotavljali zanesljivost poslovnih logik.

Z implementacijo predlaganih izboljšav - predvsem Hilt DI, razdelitev velikih komponent, navigacijski okvir, pravilna migracija baze in testno pokritje - bi se aplikacija lahko razvila iz trenutnega prototipa v robustno produkcijsko orodje za terenske monterje po vsej Sloveniji.